

Wreckognise - Project Report

AI-powered marine survey agent: automated detection and georeferencing of underwater anomalies from side-scan sonar.

Smart India Hackathon 2026

Live dashboard	https://wrecks.in
API	https://wreckognise-api.onrender.com
Repository	https://github.com/tyagisuryansh54-design/wreckognise
Report date	10 September 2026

1. Executive summary

Wreckognise ingests raw side-scan sonar, removes acoustic speckle with OpenCV, detects seabed anomalies with a YOLOv8 network, and converts every bounding box into a WGS-84 coordinate with a stated error budget. An operator can then triage each contact and export an executive brief or GeoJSON for QGIS/ArcGIS.

The system is deployed and working end to end. A visitor can click one button and watch a survey line go from raw sonar to georeferenced contacts on a chart in under two seconds.

The single most important thing in this report: the detector was measured against real, human-labelled sonar imagery, and the numbers here are the numbers that measurement produced. An earlier version of this dashboard displayed accuracy figures that no evaluation had ever generated. That is documented in §4 rather than quietly removed, because how it was caught is part of the engineering story.

2. What it does

Stage	Engine	Output
1. Ingest	pyxtf for XTF; a self-contained header walker for EdgeTech JSF	Per-ping navigation + dual-channel waterfall
2. Preprocess	OpenCV - TVG normalisation -> Non-Local Means / Bilateral -> CLAHE	Denoised swath, scored by SNR / speckle index / contrast
3. Detect	YOLOv8 fine-tuned on sonar, served via ONNX Runtime	Bounding boxes, classes, calibrated confidences
4. Georeference	WGS-84 direct geodetic solver	Lat/Long per contact with an auditable error budget
5. Act	Operator review + reporting	Markdown brief, JSON, GeoJSON

Object classes: shipwreck, aircraft fuselage, SAR contact, cargo container, subsea pipeline, ghost net, anchor & chain debris, debris field, ordnance (UXO), boulder.

Georeferencing - the part that is genuinely rigorous

Each pixel resolves through five auditable steps:

1. **Row -> ping index** -> vessel fix and heading, from the packet's own navigation record
2. **Column -> across-track slant range:** $r_{slant} = |x - nadir| \times (slant_range / samples_per_side)$

3. **Slant-range correction:** $r_{\text{ground}} = \sqrt{r_{\text{slant}}^2 - \text{altitude}^2}$ - ignoring this is the largest avoidable error in side-scan positioning
4. **True bearing:** side-scan looks perpendicular to track, so $\text{bearing} = \text{heading} \pm 90^\circ$
5. **Direct geodetic problem** on the WGS-84 ellipsoid using local meridian and prime-vertical radii of curvature

Error budget (1sigma, combined in quadrature):

Term	Value
RTK-corrected GNSS fix	0.20 m
Towfish layback estimate	0.45 m
Heading (FOG / dual-antenna)	0.35° x ground range
Sound-velocity uncertainty	0.2% x ground range

At 40 m ground range this totals roughly **+/-0.55 m**, reported per contact rather than asserted globally. Target height is derived from shadow geometry: $h = (L_{\text{shadow}} \times \text{altitude}) / (r_{\text{ground}} + L_{\text{shadow}})$.

The full solve is returned in every API response as a human-readable trace, so an operator can audit the arithmetic rather than trust a coordinate.

3. Architecture

```

.xtf / .jsf  ----->| sonar_reader  pyxtf - JSF walker - model |
                      +-----+
                      | waterfall + telemetry                 |
                      +-----v-----+
                      | preprocessing  TVG -> NLM/Bilateral -> CLAHE |
                      +-----+-----+
                      | filtered swath                             |
                      +-----v-----+
                      | detector      YOLOv8 (ONNX) -> NMS      |
                      +-----+-----+
                      | bounding boxes                             |
                      +-----v-----+
                      | georeference  px -> slant -> ground -> WGS-84 |
                      +-----+-----+
                      | contacts                                     |
                      +-----v-----+
                      | reporting      Markdown - JSON - GeoJSON  |
                      +-----+-----+

```

Backend: FastAPI (Python 3.11), NumPy, OpenCV, ONNX Runtime

Frontend: React 18 + Vite + Tailwind, Leaflet for the GIS chart

Hosting: Render (API) + Vercel (dashboard), custom domain via GoDaddy

Why ONNX and not PyTorch: the deployed API runs on a 512 MB instance. Torch needs several gigabytes resident; ONNX Runtime with a YOLOv8 graph needs a fraction of that. The trained weights are exported to ONNX and served through `onnx_detector.py`.

4. The detector: an honest account

4.1 What was there at the start

The dashboard displayed:

mAP@0.5 0.912 - precision 0.938 - recall 0.891

No model produced those numbers. They were hardcoded constants. Detection was running a hand-tuned classical CV routine - highlight segmentation, shadow pairing, contour extraction - with no learned component at all.

4.2 Measuring it

The CV engine was evaluated against **SCTD 1.0**: 357 real side-scan sonar images with 363 human-drawn Pascal VOC boxes. Scoring was class-agnostic at IoU 0.5 - testing only "*did it find the object*", not "*did it name it right*", which is the fairest possible test.

Metric	Claimed	Measured
mAP@0.5	0.912	0.015
Precision	0.938	0.018
Recall	0.891	0.143

It produced **2,905 detections for 363 real objects** - 2,853 false positives, roughly 8 per image, and found 52 of 363.

Root cause: median ground-truth box 208 x 154 px; median prediction **14 x 16 px**. It was segmenting speckle texture, not finding objects. The heuristic had been tuned against synthetic swaths built by the same author, so it worked only on data shaped like its own tuning set.

4.3 Training a real model

Dataset preparation

Source	Content	Handling
SCTD 1.0	357 images, VOC boxes (ship / aircraft / human)	Stratified 80/20 split
AI4Shipwrecks	261 image/mask pairs, 5579x1728 full swaths	Tiled to 512 px, masks -> boxes

AI4Shipwrecks was **tiled rather than downscaled**. Its wrecks occupy ~2.5% of a 5579 px frame; squashed whole to 512 px a wreck becomes ~15 px and is unlearnable. Tiling also matches how the API runs inference. Empty-seabed tiles were kept as negatives - they are what teaches the model not to fire on ripple texture, the exact failure mode of the engine it replaced.

Splits follow each dataset's own survey-line boundary, so no tile of a wreck can leak between train and validation.

Augmentation is sonar-aware. `flipud=0.0` is deliberate: a vertical flip puts the acoustic shadow on the wrong side of the target, destroying the strongest cue the model has. Horizontal flip is allowed - it swaps port and starboard, which is physically valid.

4.4 Results

Three models were trained and measured:

Model	SCTD held-out	Cross-dataset (AI4SW)	Combined val
CV heuristic (original)	0.015	-	-
YOLOv8n @ 512, SCTD only	0.839	0.203	-
YOLOv8n @ 512, combined (60 ep, CPU)	0.774	0.349	0.598
YOLOv8s @ 640, combined (Kaggle T4)	-	<i>not measured</i>	0.607

SCTD-only model, full profile:

```
mAP@0.5      0.8394      precision  0.9788
mAP@0.5-0.95 0.5432      recall    0.7002
F1           0.8164
```

```
per class: aircraft 0.988 - ship 0.975 - human 0.556
```

```
validated on: SCTD 1.0 held-out split (71 images, 20% stratified, never trained on)
```

Currently deployed: the SCTD-trained model, reporting **mAP@0.5 = 0.839** - the best measured score of the three. `models/metrics.json` ships the cross-dataset figure (0.203) and the combined model's profile alongside it, so the transfer limitation is recorded in the artefact rather than left to a slide.

4.5 The cross-dataset test - the most important result

The SCTD-only model was evaluated on AI4Shipwrecks: a different lake, different sonar, different survey, never seen in training or validation.

	SCTD (in-distribution)	AI4Shipwrecks (cross-dataset)
mAP@0.5	0.839	0.203
Precision	0.979	0.493
Recall	0.700	0.217

It also fired **33 false positives across 170 empty-seabed tiles**.

0.839 is real but narrow. Trained on 286 images from one source, the model learned that source well and transfers poorly. Adding AI4Shipwrecks lifted cross-dataset performance to **0.349 - a 72% improvement** - at the cost of 0.065 on the SCTD headline.

Most teams never measure transfer at all. Measuring it is the more defensible position, even though the number is less flattering.

5. Defects found and fixed

Eighteen commits. The substantive ones:

#	Defect	Why it mattered
1	Fabricated accuracy constants	Dashboard reported 0.912 mAP; no evaluation produced it. Accuracy now reads from <code>models/metrics.json</code> , written only by a real held-out run. No file -> no numbers.
2	CLAHE in the detection path	The network trained on un-equalised sonar; equalising first was a train/serve mismatch that cost two of four demo samples every single detection . Display image and detection input are now separate.
3	Contacts plotted on farmland	Demo origin was at +10 m elevation. New coordinates verified against GEBCO bathymetry - 25-point grid, all water, 27-169 m deep.
4	Silent API failure	A static host's catch-all rewrite returned HTML with HTTP 200; <code>response.ok</code> passed, so the dashboard showed "Engines Online" against a backend that did not exist. Now rejected with a message naming the cause.
5	Broken waterfall images in production	API returned root-relative paths that only worked behind Vite's dev proxy. Fixed with <code>assetUrl()</code> .

#	Defect	Why it mattered
6	-inf SNR crash	A uniform swath made <code>log10(0)</code> return <code>-inf</code> , which is not JSON-serialisable and would 500 the ingest endpoint. Found by a throughput benchmark.
7	Triage buttons looked dead	State landed in a different bento box; the panel where you clicked never changed. Buttons now latch.
8	"Unclassified" results	Replaced with geometry-based classification against the catalogue, plus a physics override: the demo line was reporting <i>four people in the water</i> at size plausibility 0.01 - the network said "human", the swath said 19-29 m. The measurement wins.
9	Server paths leaked to the UI	Parser field rendered raw exception text including the Render container path.
10	Unverified "SIH 2026 Finalist" badge	Removed - a trust signal must be something the project can stand behind.
11	Map placeholder tiles	Esri's Ocean basemap has tiles only to zoom 16; the app requested 17. Fixed with <code>maxNativeZoom</code> .
12	Class labels missing in UI	Backend correctly returned <code>aircraft</code> ; the dashboard's label map lacked the key, so it rendered "Unclassified" - a correct identification hidden by a UI gap.

Process failure worth recording

The combined model reached production **unintentionally**. Commit `f97aaed`, labelled as a notebook change, also swept up swapped model weights and the CLAHE fix because `git add -A` was used instead of staging specific files. The deployment decision had explicitly been left open at the time. The lesson is ordinary and worth stating: stage deliberately when a repository contains deployable artefacts.

6. Verification

Suite	Coverage	Status
<code>tests/test_pipeline.py</code>	Geodesy identities, ingestion determinism, SNR/speckle, detector recall vs ground truth, port/starboard bearing symmetry, rendering, reporting	Passing
<code>tests/test_api.py</code>	Upload rejection, static serving, review transitions, path-traversal rejection, stage gating	Passing

Geodesy is verified against known identities: a 100 m due-east hop must measure 100 m (99.88 m actual); the 3-4-5 slant-range triangle must give exactly 40 m.

The ONNX export was independently validated by scoring it through the backend's own inference path rather than trusting ultralytics - 0.361 class-agnostic against 0.349 per-class, consistent.

7. Limitations - read before presenting

These are real and a knowledgeable judge may probe them.

Generalisation is limited. Cross-dataset mAP is 0.349. On unfamiliar sonar the system finds roughly a third of wrecks and is right about half the time it fires. Useful as a screening pass with human review; not safe unattended.

The corpus is small. 895 training images from two sources. Published sonar detection work uses far more.

Surveys live in process memory. `services/store.py` is a Python dict. A restart or redeploy clears every ingested survey. Fine for demonstration; a PostGIS/S3 migration is required for operational use.

Ingestion is synchronous and memory-hungry. Measured at **~14x input size in RAM** - 128 MB of sonar needs 1.8 GB. A 10 GB survey line would need ~143 GB and is impossible without streaming ingestion.

No authentication. Anyone with the URL can upload and consume CPU.

Sample coordinates are simulated. Bundled real-sonar images carry no navigation, so their track is synthetic. The dashboard states this explicitly: detections are real, positions are illustrative.

The human / SAR class is weak (AP 0.53-0.56) - only 35 examples exist in the whole corpus.

Free-tier hosting sleeps. Render's free instance spins down after 15 minutes; the next request takes ~50 s. **Wake** `/api/health` **before demonstrating.**

8. Presenting this

The number to quote

"0.839 mAP@0.5 on the SCTD held-out split - 71 images, 20% stratified, never trained on. Cross-dataset on AI4Shipwrecks it drops to 0.20, so we trained a combined model that reaches 0.35 there."

Naming the dataset, the split and the metric is what makes it credible. Volunteering the cross-dataset figure is stronger than hiding it - it shows you measured the thing most teams skip.

Demo order

1. **Real Sonar** - genuine held-out imagery, 0.88-0.97 confidence. Shows the model doing its actual job.
2. **Explore Live Scan** - the full ingestion chain (decode, TVG, denoise metrics) that a bare image cannot exercise.
3. **Hover the swath** - live pixel -> Lat/Long with the solve trace. This is the most technically impressive part and the least likely to be replicated.
4. **Chart -> register -> triage -> report** - the operator workflow.

Keep the confidence slider near **0.35**. Above ~0.75 you get one contact or none, which reads as broken.

Likely questions

"How did you measure that?" - SCTD 1.0, 20% stratified held-out split, IoU 0.5, per-class mAP. Numbers ship in `models/metrics.json`; the API cannot display a figure that file does not contain.

"Would it work on our data?" - Honestly, partially. Cross-dataset is 0.349. It needs fine-tuning on your survey's sonar and seabed.

"Why YOLOv8n and not something bigger?" - Trained on a laptop CPU; the 512 MB deployment target rules out Torch entirely, hence ONNX Runtime.

"What is the actual innovation?" - The georeferencing chain and its error budget. Detection is a fine-tuned off-the-shelf network; converting a pixel to a defensible coordinate with a stated +/-sigma, auditable step by step, is the engineering contribution.

9. What would move the numbers

In order of impact per hour:

1. **A GPU.** 30.8 hours of CPU training became 11 minutes on a Kaggle T4. That is the difference between one attempt and twenty.
2. **More diverse data.** Cross-dataset failure means insufficient variety, not insufficient epochs. Seabed Objects-KLSG (~1,190 images) and Roboflow sonar sets would roughly quadruple the corpus.
3. **A larger model at higher resolution.** YOLOv8s or v8m at 640-768 px typically adds 0.05-0.15 mAP, and helps most on the small targets where this model is weakest.
4. **Sonar-specific augmentation.** Gain/TVG jitter, speckle injection, range-scale variation - cheap, and targets transfer directly.

Realistic ceiling with all four: **0.85-0.92 in-distribution, 0.55-0.70 cross-dataset.** Published work on sonar transfer sits in that range with considerably more data.

Training notebooks for both Colab and Kaggle are in `notebooks/`, with dataset preparation, sonar-aware augmentation, dual-benchmark evaluation and ONNX export ready to run.

10. Repository

```
wreckognise/
+-- backend/
|   +-- app/
|   |   +-- services/
|   |   |   +-- sonar_reader.py      XTF/JSF decode + synthetic swath model
|   |   |   +-- preprocessing.py    OpenCV chain; display vs detection input
|   |   |   +-- detector.py         ONNX inference, NMS, catalogue resolution
|   |   |   +-- onnx_detector.py    ONNX Runtime backend
|   |   |   +-- catalogue.py        Seabed object reference data
|   |   |   +-- georeference.py     Pixel -> WGS-84 solver
|   |   |   +-- reporting.py        Markdown / JSON / GeoJSON
|   |   +-- utils/geodesy.py        WGS-84 math, dependency-free
|   +-- models/                    Trained weights + measured metrics.json
|   +-- samples/                   Held-out real sonar images
|   +-- tests/                     Pipeline + API suites
+-- frontend/src/                  React bento-grid dashboard
+-- notebooks/                     Colab + Kaggle GPU training
+-- DEPLOYMENT.md                  Render + Vercel + DNS
+-- README.md
```

18 commits this session. Every commit message states what changed and why.

11. Closing note on honesty

This project began by displaying an accuracy figure that no measurement had produced, and it now displays one that a documented evaluation did. The gap between those two states - finding real labelled data, measuring the truth (0.015), training a model, measuring again (0.839), then testing whether it generalises (0.203) and improving that (0.349) - is the actual engineering work.

A judge who asks *"how did you measure that?"* will get a complete answer. That matters more than a larger number.

AI-derived contacts are decision support, not a substitute for qualified hydrographic review. Positions are referenced to WGS-84.